

目前已学习内容

2.26-3.18，从基础的理论知识开始学习，完成了：

1. **正点原子Linux驱动文档的前三期内容。** 视频课太长太浪费时间就没看，看的对应文档，重点看了第三篇uboot的移植以及uboot和Linux kernel的启动流程，还有shell的基本操作命令和makefile文件的基础语法，第二期当时重点看了比较重要外设的原理，其余的都没看，代码和操作的部分直接跳过了，不过现在完全想不起来看过的内容了（悲）；第四期是Linux驱动，是整篇文档最核心的部分，不过当时决定先找一个内核和操作系统相关的项目做做，这部分就先搁置吧，如果后面要做驱动的项目时再作为重点学习。
2. **b站王道的《操作系统》课程。** 进程和线程以及内存管理作为重点认真看了一遍，理解了几个重要的概念比如cpu调度的原理和虚拟文件虚拟地址的概念；后面文件系统也是跳着看了看，越到后面越看不下去了，感觉看课看到后面还是有点煎熬，，io设备管理好像也没有那么重要？不管了，用到了再学吧！
3. **b站《开发一个RISC-V上的操作系统》课程。** 课程前半段主要是铺垫，补充一些基础知识，比如计算机底层的一些原理，ISA的详细介绍，riscv的汇编语法，还有编译和链接的整个过程，有些概念在前面的学习中也已经涉及到了，听起来还是比较轻松，但是看完有点脑子空空了，riscv的汇编语法也只记得最简单的几个。不过问题不大，后面做项目不会的东西再返回来复习；后半部分进入实战环节，正式开始构建一个操作系统，看了几节课能听懂个大概，但是有些内容听的云里雾里的，再加上最近一直看课，实在坚持不太下去了，迫不及待想开始实践项目了，，后面的打算不看了。

现在正在复现一个操作系统相关的项目，进度35%，如下：

2023.6 - 2023.10

基于 Qemu 创建嵌入式 Riscv64 操作系统

简易操作系统内核实现

项目描述：此项目为 rCore 的 C 语言重置版，在 Qemu 虚拟机上自定义了一块 RISCv64 的 SOC，并基于此虚拟硬件编写了一个简单操作系统内核

技术栈：C、Rust、Qemu、OpenSBI、RISCv

工作内容：

- 使用 Qemu 的内置接口定义了一块 RV64 的 SOC，包含 Flash、PLIC、CLINT、RTC、UART 等硬件
- 为 SOC 编写设备树文件，移植 OpenSBI，编写 boot 代码在 Flash 运行用于加载固件到 DRAM 处执行
- 使用栈式数据结构来管理和分配每一页物理内存，基于 RTC 时钟中断实现了分时多任务调度机制
- 基于 RISCv 的 sv39 分页机制实现了各用户态进程和内核的虚实地址映射，内核采用恒等映射方式
- 设计了一种应用加载机制，遍历进程二进制文件夹，生成进程的汇编符号，通过 incbin 加载进程数据
- 实现了常用系统调用如 fork、exec、read、write 等，在系统启动时会拉起 user_shell 进程

- 对 cpu 核心和内存范围进行了安全域与非安全域的划分，在安全域移植了 FreeRTOS 运行

项目地址：<https://github.com>